

# Build a To-Do App

Next.js · TypeScript · Node.js · PostgreSQL · Zustand: a beginner's study guide

This guide walks you through a small but **real** full-stack app: a page where you add, tick and delete tasks, saved in a PostgreSQL database. It's about 150 lines of code in 7 files, small enough to memorize and big enough to teach the things LeetCode can't:

- how a browser talks to a server (**HTTP, APIs, JSON**)
- how data is stored and queried (**databases, SQL**)
- how a project is organised (**folders, config, secrets**)
- how a UI keeps track of data (**state, Zustand**)

Every file is shown with a **line-by-line explanation** table: code on the left, meaning on the right. All commands are for **Windows PowerShell**.

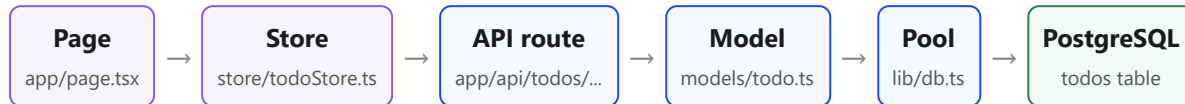
|               |                                                                                         |
|---------------|-----------------------------------------------------------------------------------------|
| <b>Part 1</b> | Understand before you type: the big picture, key words, folders, who creates which file |
| <b>Part 2</b> | Set up PostgreSQL: install, psql, create the database and table                         |
| <b>Part 3</b> | Create the Next.js project with one command, add libraries                              |
| <b>Part 4</b> | Write the code, back to front: database → model → API → store → page                    |
| <b>Part 5</b> | Compare with Express + Mongoose                                                         |
| <b>Part 6</b> | Memorize and practice, plus troubleshooting                                             |

# Part 1: Understand before you type

Ten minutes here saves hours of confusion later.

## The big picture

When you click **Add**, your task travels through these pieces and back:



Purple = runs in the **browser**. Blue = runs on the **server** (Node.js). Green = the **database**.

### IN PLAIN ENGLISH

Think of a restaurant:

| Restaurant                   | Our app         | Job                                                                           |
|------------------------------|-----------------|-------------------------------------------------------------------------------|
| Dining room                  | Page            | What the customer sees and clicks.                                            |
| Order pad                    | Store (Zustand) | Remembers the current list; sends orders to the kitchen.                      |
| Waiter at the kitchen window | API route       | Takes an order (request), checks it makes sense, returns the dish (response). |
| Cook                         | Model           | Knows the recipes, i.e. the SQL for each job.                                 |
| Phone lines to the pantry    | Pool            | A few always-open lines to the database, shared by everyone.                  |
| Pantry                       | PostgreSQL      | Where everything is stored permanently.                                       |

## Words you'll see

| Word           | Meaning                                                                                                                            |
|----------------|------------------------------------------------------------------------------------------------------------------------------------|
| Node.js        | Runs JavaScript outside the browser, on your computer or a server. Next.js runs on it.                                             |
| npm            | Node's package manager. Downloads libraries into <code>node_modules/</code> and lists them in <code>package.json</code> .          |
| Next.js        | A framework that serves your <b>pages</b> and your <b>API</b> from one project. It replaces Express here.                          |
| React          | The library for building the UI out of <b>components</b> (functions that return HTML-like JSX).                                    |
| TypeScript     | JavaScript plus <b>types</b> . Catches mistakes (like a typo in a field name) before you run the code.                             |
| State          | Data that, when it changes, makes the screen redraw. <code>useState</code> (one component) or a store (shared).                    |
| Hook           | A React function starting with <code>use...</code> ( <code>useState</code> , <code>useEffect</code> , <code>useTodoStore</code> ). |
| Zustand        | A tiny library for <b>shared state</b> : one store any component can read and update.                                              |
| API / endpoint | A URL that returns <b>data</b> (JSON) instead of a page, e.g. <code>/api/todos</code> .                                            |
| HTTP method    | The verb of a request: <b>GET</b> read, <b>POST</b> create, <b>PATCH</b> update, <b>DELETE</b> remove.                             |

|                      |                                                                                                                                   |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Status code          | A number in each reply: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 404 Not Found, 500 Server Error.                    |
| JSON                 | Text format for data: <code>{"id": 1, "text": "Buy milk", "done": false}</code> .                                                 |
| PostgreSQL           | A <b>relational</b> database: data lives in <b>tables</b> with fixed <b>columns</b> ; each item is a <b>row</b> .                 |
| SQL                  | The language for talking to the database: <code>SELECT</code> , <code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code> . |
| psql                 | PostgreSQL's terminal app for typing SQL. Like <code>mongosh</code> for MongoDB.                                                  |
| pg                   | The npm library your Node.js code uses to talk to PostgreSQL.                                                                     |
| Connection pool      | A few database connections kept open and shared, so each request doesn't open a new one.                                          |
| Environment variable | A setting kept outside the code (in <code>.env.local</code> ), e.g. the database password.                                        |

## The folders: is this the standard design?

### project structure

```

todo-app/
├─ app/                ← REQUIRED by Next.js: pages and API routes
│  ├─ layout.tsx       ← frame around every page
│  ├─ page.tsx         ← the page at /
│  └─ api/todos/
│     ├─ route.ts       ← API: GET, POST /api/todos
│     └─ [id]/route.ts  ← API: PATCH, DELETE /api/todos/7
├─ lib/db.ts           ← convention: shared helpers (database pool)
├─ models/todo.ts      ← convention: all the SQL for todos
├─ store/todoStore.ts  ← convention: Zustand store
├─ schema.sql          ← the table definition (run once with psql)
├─ .env.local          ← secrets: database address + password
└─ package.json, tsconfig.json, next.config.ts ... ← config (generated)

```

| Folder  | Required? | Why it's there                                                                                                                                                                                                |
|---------|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| app/    | Yes       | Next.js's <b>App Router</b> turns this folder into URLs. <code>page.tsx</code> = a page, <code>route.ts</code> = an API endpoint, <code>[id]</code> = a changing part of the URL. Those file names are fixed. |
| lib/    | No        | Very common convention for helpers and setup code. You could call it <code>utils/</code> ; Next.js doesn't care.                                                                                              |
| models/ | No        | Our choice: puts all SQL in one place, like Mongoose models. Keeps API routes short.                                                                                                                          |
| store/  | No        | Our choice: where Zustand stores live.                                                                                                                                                                        |

### IN PLAIN ENGLISH

Next.js only cares about `app/` (and `public/` for images, which we don't use). Everything else is a tidy habit. Other projects often add `components/` , `hooks/` , `types/` , or put everything inside `src/` . All of those are normal.

## Who creates which file?

Many files appear "by magic". Here is where each one comes from and whether you touch it:

| File / folder                                                | Created by                                                            | You edit it?   | What it is                                                                                                                                                                                                                                      |
|--------------------------------------------------------------|-----------------------------------------------------------------------|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| package.json                                                 | create-next-app                                                       | Rarely         | Project name, scripts ( <code>npm run dev</code> ), list of libraries.                                                                                                                                                                          |
| package-lock.json                                            | npm install                                                           | Never          | Exact versions of every library, so installs are repeatable.                                                                                                                                                                                    |
| node_modules/                                                | npm install                                                           | Never          | The downloaded libraries (Next.js, React, pg...). Huge; never commit it.                                                                                                                                                                        |
| tsconfig.json                                                | create-next-app                                                       | No             | TypeScript settings. (If missing, <code>next dev</code> creates it. <code>npm init -y</code> does <b>not</b> ; that only makes <code>package.json</code> .)                                                                                     |
| next.config.ts                                               | create-next-app                                                       | Later          | Next.js settings. Empty = defaults.                                                                                                                                                                                                             |
| next-env.d.ts                                                | Next.js                                                               | Never          | Tells TypeScript about Next.js's types.                                                                                                                                                                                                         |
| .git/                                                        | create-next-app<br>(if Git is installed)                              | Never directly | Git's history of your project. Use <code>git status</code> , <code>git add</code> and <code>git commit</code> instead of touching it.                                                                                                           |
| .next/                                                       | create-next-app, then <code>next dev</code> / <code>next build</code> | Never          | Next.js's <b>output and cache</b> : your code compiled for the browser and server. It is <i>not</i> the npm package (that's <code>node_modules/next</code> ); it's what the package <i>produces</i> when it runs. Safe to delete; it's rebuilt. |
| tsconfig.tsbuildinfo                                         | TypeScript                                                            | Never          | Cache that makes type-checking faster.                                                                                                                                                                                                          |
| .gitignore                                                   | create-next-app                                                       | Rarely         | Files Git should never save ( <code>node_modules</code> , <code>.next</code> , <code>.env*</code> ).                                                                                                                                            |
| AGENTS.md , CLAUDE.md , README.md                            | create-next-app                                                       | Optional       | Notes for humans and AI coding assistants. Not used by the app.                                                                                                                                                                                 |
| app/layout.tsx , app/page.tsx                                | create-next-app                                                       | <b>Yes</b>     | Generated as a "Hello world", then you replace them.                                                                                                                                                                                            |
| .env.local , schema.sql , lib/ , models/ , store/ , app/api/ | <b>You</b>                                                            | <b>Yes</b>     | Your actual app.                                                                                                                                                                                                                                |
| guide/                                                       | -                                                                     | -              | <b>Not part of the app.</b> <code>build-guide.mjs</code> and <code>explanations.mjs</code> only generate this PDF. You can delete the folder and the app still works.                                                                           |

## Part 2: Set up PostgreSQL

Install the database, meet psql, create the database and the table.

---

### 1 Install Node.js

Download the LTS version from **nodejs.org** and install it. Check it in PowerShell:

```
PowerShell
```

```
node -v  
npm -v
```

### 2 Install PostgreSQL

PostgreSQL is a program that runs in the background (a Windows **service**) and waits for connections on **port 5432**. Install it with the normal setup wizard. Start it either way:

- **From the website:** download the Windows installer from **postgresql.org/download/windows** and run it.
- **From the terminal:** this downloads the same installer and opens the same wizard:

```
PowerShell
```

```
winget install PostgreSQL.PostgreSQL.17 --interactive
```

In the wizard, click Next through everything, except:

| Screen      | What to do                                                                                                                                                   |
|-------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Password    | Choose a password for the main database user, called <code>postgres</code> . Letters and numbers are easiest. <b>Write it down</b> ; you need it in step 10. |
| Port        | Keep <b>5432</b> .                                                                                                                                           |
| Last screen | Untick "Stack Builder"; you don't need it.                                                                                                                   |

#### WATCH OUT

Windows will show an **administrator (UAC) prompt**; click Yes. It may hide behind other windows (look for a flashing shield on the taskbar). Download plus install can take 10 to 15 minutes.

### FULLY AUTOMATIC INSTALL (OPTIONAL)

To install with **no** windows at all (for scripts, or when someone installs it for you), pass the wizard's answers up front. That's why this command is so long:

PowerShell

```
winget install --id PostgreSQL.PostgreSQL.17 --source winget --override "--mode unattended --unattendedmodeui none --superpassword YOUR_PASSWORD --serverport 5432"
```

`--override` hands the rest straight to the installer: `--mode unattended --unattendedmodeui none` = no windows, `--superpassword` = the password answer, `--serverport` = the port answer. This is how this project's database was installed; the password went straight into `.env.local`.

### Check that it's running

PowerShell

```
sc.exe query postgresql-x64-17
```

Look for `STATE : 4 RUNNING`. It starts automatically every time Windows starts. The installer also adds **pgAdmin 4**, a point-and-click app for browsing your data (like MongoDB Compass).

## 3 Meet psql

### IN PLAIN ENGLISH

**psql** is a terminal "chat window" with your database: you type SQL or commands, it answers. It's the PostgreSQL version of `mongosh`. You use it to set things up and peek at data; your **app** doesn't use `psql`, it uses the `pg` library.

`psql` lives in `C:\Program Files\PostgreSQL\17\bin`. Windows doesn't know that folder yet, so typing `psql` gives "not recognized". Fix it once by adding the folder to your **PATH**:

1. Press Start, type **environment variables**, open "Edit the system environment variables".
2. Click **Environment Variables...**, select **Path** under "User variables", then **Edit**, then **New**.
3. Paste `C:\Program Files\PostgreSQL\17\bin`, click OK three times, then **open a new PowerShell window**.

(Without PATH, you can always type the full path: `& "C:\Program Files\PostgreSQL\17\bin\psql.exe"` ...)

The flags you'll use:

| Flag            | Meaning                    | Our value                           |
|-----------------|----------------------------|-------------------------------------|
| <code>-U</code> | User to log in as          | <code>postgres</code>               |
| <code>-h</code> | Host (which computer)      | <code>localhost</code> (this one)   |
| <code>-d</code> | Database to open           | <code>todo_app</code>               |
| <code>-c</code> | Run one command and exit   | <code>"SELECT * FROM todos;"</code> |
| <code>-f</code> | Run all commands in a file | <code>schema.sql</code>             |

**TIP**

psql asks for the password every time. To skip that in the current window only, run `$env:PGPASSWORD = "YOUR_PASSWORD"` first. It's forgotten when you close the window.

## 4 Create the database

One PostgreSQL server can hold many databases (one per project). Create ours:

PowerShell

```
psql -U postgres -h localhost -c "CREATE DATABASE todo_app;"
```

It answers `CREATE DATABASE`. Do this once; running it again says "already exists", which is harmless.

## 5 Create the table

Unlike MongoDB, PostgreSQL needs to know the **shape** of your data before you store any: a table with named, typed columns. Save this as `schema.sql` in your project folder (you'll create the folder in step 6, so you can also come back to this step):

schema.sql

| Code                                  | What it means                                                                                                                                                                                          |
|---------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| -- Run this once to create the table. | A comment. In SQL, comments start with <code>--</code> . The database ignores it.                                                                                                                      |
| CREATE TABLE IF NOT EXISTS todos (    | Make a new table called <code>todos</code> . <code>IF NOT EXISTS</code> means: if it's already there, do nothing instead of crashing.                                                                  |
| id SERIAL PRIMARY KEY,                | Column 1: <code>id</code> . <code>SERIAL</code> = Postgres fills in 1, 2, 3... for you. <code>PRIMARY KEY</code> = no two rows can share an id. (Like Mongo's <code>_id</code> , but a simple number.) |
| text TEXT NOT NULL,                   | Column 2: <code>text</code> , the task name. <code>TEXT</code> = any length string. <code>NOT NULL</code> = it must always have a value.                                                               |
| done BOOLEAN NOT NULL DEFAULT FALSE   | Column 3: <code>done</code> , true or false. If you don't give one, it starts as <code>FALSE</code> .                                                                                                  |
| );                                    | Close the column list. The <code>;</code> ends the SQL command.                                                                                                                                        |

Run the file against the `todo_app` database, then check the result with `\d` (describe):

PowerShell

```
psql -U postgres -h localhost -d todo_app -f schema.sql
psql -U postgres -h localhost -d todo_app -c "\d todos"
```

#### expected output

| Column | Type    | Nullable | Default                           |
|--------|---------|----------|-----------------------------------|
| id     | integer | not null | nextval('todos_id_seq'::regclass) |
| text   | text    | not null |                                   |
| done   | boolean | not null | false                             |

Indexes:

"todos\_pkey" PRIMARY KEY, btree (id)

#### Handy psql commands (inside `psql -U postgres -h localhost -d todo_app`)

| Type this                         | It does                                          | Mongo shell equivalent        |
|-----------------------------------|--------------------------------------------------|-------------------------------|
| <code>\l</code>                   | List databases                                   | <code>show dbs</code>         |
| <code>\c todo_app</code>          | Switch database                                  | <code>use todo_app</code>     |
| <code>\dt</code>                  | List tables                                      | <code>show collections</code> |
| <code>\d todos</code>             | Show a table's columns                           | (no schema in Mongo)          |
| <code>SELECT * FROM todos;</code> | Show all rows (don't forget the <code>;</code> ) | <code>db.todos.find()</code>  |
| <code>\q</code>                   | Quit                                             | <code>exit</code>             |



## Part 3: Create the Next.js project

One command makes the skeleton; two more add our libraries.

### 6 Generate the project (boilerplate)

Go to the folder where you keep projects, then run:

PowerShell

```
npx create-next-app@latest todo-app --ts --app --empty --no-tailwind --no-eslint --no-src-dir --no-react-compiler --import-alias "@/*" --use-npm
cd todo-app
```

| Part                                                       | Meaning                                                                                      |
|------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| <code>npx create-next-app@latest todo-app</code>           | Download and run Next.js's official project generator; make a folder <code>todo-app</code> . |
| <code>--ts</code>                                          | Use TypeScript.                                                                              |
| <code>--app</code>                                         | Use the App Router (the <code>app/</code> folder).                                           |
| <code>--empty</code>                                       | Minimal starter: just a "Hello world!" page, no demo images or CSS.                          |
| <code>--no-tailwind --no-eslint --no-react-compiler</code> | Skip extra tools, to keep it simple for learning.                                            |
| <code>--no-src-dir</code>                                  | Put <code>app/</code> at the top level, not inside <code>src/</code> .                       |
| <code>--import-alias "@/*"</code>                          | Make <code>@/</code> mean "project root" in imports.                                         |
| <code>--use-npm</code>                                     | Use npm (not yarn/pnpm/bun).                                                                 |

#### TIP

Prefer questions? Run just `npx create-next-app@latest todo-app` and pick custom settings that match the flags above: TypeScript **yes**, App Router **yes**, Tailwind / ESLint / `src/` **no**, import alias `@/*`.

#### IN PLAIN ENGLISH

**Know Vite?** This is the same idea as `npm create vite@latest my-app -- --template react-ts`: one command makes a ready-to-run starter. The difference is that Vite gives you a browser-only React app (you'd still need a separate backend like Express), while Next.js gives you pages *and* API routes in one project.

**What it creates** (this project was made with exactly this command):

created by create-next-app

```
todo-app/
├ .git/                ← a Git repository with one commit (if Git is installed)
├ .next/              ← generated route types (more appears when you run the app)
├ app/
│   ├── layout.tsx     ← you'll replace this
│   └── page.tsx       ← "Hello world!", you'll replace this
├ node_modules/       ← next, react, react-dom, typescript, @types/...
├ .gitignore
├ AGENTS.md, CLAUDE.md, README.md
├ next-env.d.ts
├ next.config.ts
├ package.json
├ package-lock.json
└ tsconfig.json
```

It also runs `npm install` for you and, if Git is installed, makes a first commit called "Initial commit from Create Next App". From then on, `git status` shows exactly which files **you** changed or added.

## 7 Add our three libraries

PowerShell

```
npm install pg zustand
npm install -D @types/pg
```

| Package   | What for                                                                                                                                                        |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| pg        | Lets Node.js talk to PostgreSQL.                                                                                                                                |
| zustand   | Shared state in the browser.                                                                                                                                    |
| @types/pg | Type descriptions for <code>pg</code> , so TypeScript understands it. <code>-D</code> = a <b>dev</b> -dependency: only needed while coding, not to run the app. |

### IN PLAIN ENGLISH

**Why isn't the install command long anymore?** Without create-next-app you'd have to install everything by hand: `npm install next react react-dom pg zustand` plus `npm install -D typescript @types/node @types/react @types/react-dom @types/pg`. Every `@types/...` package teaches TypeScript about one JavaScript library. create-next-app already installed all of them except `@types/pg`, so that's all you add.

Some libraries ship their own types and need no `@types` package. `zustand` is one; `pg` isn't.

These commands change `package.json` (new lines under `dependencies` / `devDependencies`), `package-lock.json` and `node_modules/`.

## 8 Run it once

PowerShell

```
npm run dev
```

Open **http://localhost:3000** and you'll see "Hello world!". Next.js just filled the `.next/` folder with the compiled app. Leave it running: it reloads automatically as you save files. Press **Ctrl+C** to stop it.

## 9 Understand the generated config (no changes needed)

You won't edit these, but you should know what they say.

| tsconfig.json                                  |                                                                                                                                                            |
|------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Code                                           | What it means                                                                                                                                              |
| {                                              | This file is one JSON object of TypeScript settings. <b>create-next-app wrote it for you; you never need to change it.</b>                                 |
| "compilerOptions": {                           | Everything inside this block tells the TypeScript checker how to behave.                                                                                   |
| "target": "ES2017",                            | Which JavaScript version to aim for. Next.js handles old browsers itself, so this rarely matters.                                                          |
| "lib": ["dom", "dom.iterable", "esnext"],      | Which built-in type definitions to load: browser things ( <code>document</code> , <code>fetch</code> ) and modern JavaScript.                              |
| "allowJs": true,                               | Plain <code>.js</code> files are allowed in the project too.                                                                                               |
| "skipLibCheck": true,                          | Don't type-check the insides of downloaded libraries. Faster, and avoids errors that aren't yours.                                                         |
| "strict": true,                                | Turn on all the strict safety checks. This is the whole point of TypeScript: catching mistakes like using something that might be <code>undefined</code> . |
| "noEmit": true,                                | TypeScript only <b>checks</b> ; it doesn't produce any files. Next.js does the real compiling.                                                             |
| "esModuleInterop": true,                       | Lets <code>import</code> work smoothly with older libraries (like <code>pg</code> ).                                                                       |
| "module": "esnext",                            | Use modern <code>import</code> / <code>export</code> syntax.                                                                                               |
| "moduleResolution": "bundler",                 | Find imported files the way modern bundlers (like Next.js) do.                                                                                             |
| "resolveJsonModule": true,                     | You may <code>import</code> a <code>.json</code> file.                                                                                                     |
| "isolatedModules": true,                       | Every file must make sense on its own. Next.js's fast compiler needs this.                                                                                 |
| "jsx": "react-jsx",                            | How to treat the HTML-looking <code>&lt;div&gt;</code> code in <code>.tsx</code> files. You don't need to import React in every file.                      |
| "incremental": true,                           | Remember the last check so the next one is faster. That's what the <code>tsconfig.tsbuildinfo</code> cache file is.                                        |
| "plugins": [<br>{<br>"name": "next"<br>}<br>], | Turns on the Next.js helper inside your editor (better autocomplete and warnings).                                                                         |
|                                                |                                                                                                                                                            |

|                                                                                                                                             |                                                                                                                                                                                                                                           |
|---------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>"paths": {   "@/*": ["./*"] } },</pre>                                                                                                 | <p>A shortcut: <code>@/</code> means "the project's root folder". So <code>import { pool } from "@/lib/db"</code> works from any folder, with no <code>../..</code>. The <code>}</code> after it closes <code>compilerOptions</code>.</p> |
| <pre>"include": [   "next-env.d.ts",   "**/*.ts",   "**/*.tsx",   ".next/types/**/*.ts",   ".next/dev/types/**/*.ts",   "**/*.mts" ],</pre> | <p>Which files to check: every <code>.ts</code> / <code>.tsx</code> file, plus types that Next.js generates inside <code>.next/</code>.</p>                                                                                               |
| <pre>"exclude": ["node_modules"] }</pre>                                                                                                    | <p>Never check <code>node_modules</code> (the downloaded libraries).</p>                                                                                                                                                                  |

| next.config.ts                                                             |                                                                                                                                                                                           |
|----------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Code                                                                       | What it means                                                                                                                                                                             |
| <pre>import type { NextConfig } from "next";</pre>                         | <p>Bring in the <i>type</i> <code>NextConfig</code>, so your editor can autocomplete valid settings. ( <code>import type</code> = only used for checking, removed when the app runs.)</p> |
| <pre>const nextConfig: NextConfig = {   /* config options here */ };</pre> | <p>The settings object. It's empty, which means "use all the defaults". This is where you'd later add things like image domains or redirects.</p>                                         |
| <pre>export default nextConfig;</pre>                                      | <p>Hand the settings to Next.js. It reads this file when <code>npm run dev</code> starts.</p>                                                                                             |

## Part 4: Write the code

Back to front: secret → database connection → model → API → store → page.

### 10 Store the database address in .env.local

Create a file named exactly `.env.local` in the project root. Use the password from step 2:

| .env.local                                                                        |                                                                                                                                                                                                                                                                                                                                                 |
|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Code                                                                              | What it means                                                                                                                                                                                                                                                                                                                                   |
| <pre>DATABASE_URL=postgres://postgres:YOUR_PASSWORD@localhost:5432/todo_app</pre> | <p>One setting, written as an address:</p> <pre>postgres://USER:PASSWORD@HOST:PORT/DATABASE</pre> <p>Read it as: log in as user <code>postgres</code> with your password, on this computer ( <code>localhost</code> ), port <code>5432</code> , database <code>todo_app</code> . Code reads it with <code>process.env.DATABASE_URL</code> .</p> |

#### IN PLAIN ENGLISH

Next.js loads `.env.local` automatically when it starts, and only server code can read it, so the password never reaches the browser. `.gitignore` already lists `.env*` , so it won't be uploaded to GitHub. **If you change this file, restart `npm run dev` .**

### 11 Connect to the database: lib/db.ts

Create a folder `lib` and this file:

| lib/db.ts                                                                                                                                                                         |                                                                                                                                                                                                                                                                                |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Code                                                                                                                                                                              | What it means                                                                                                                                                                                                                                                                  |
| <pre>import { Pool } from "pg";</pre>                                                                                                                                             | Bring in <code>Pool</code> from the <code>pg</code> library (the Node.js driver for PostgreSQL).                                                                                                                                                                               |
| <pre>// A Pool keeps a few database connections open and lends one to each query. // DATABASE_URL comes from .env.local (Next.js loads it automatically, server-side only).</pre> | Comments. Read the pool deep-dive below for the full story.                                                                                                                                                                                                                    |
| <pre>function createPool() {   return new Pool({ connectionString:     process.env.DATABASE_URL }); }</pre>                                                                       | A small function that builds a new pool. The only setting is the database address from <code>.env.local</code> . Everything else uses <code>pg</code> 's defaults (up to 10 connections).                                                                                      |
| <pre>// In development, Next.js re-runs this file every time you save. // Parking the pool on globalThis lets us reuse it instead of opening new connections each time.</pre>     | Comments explaining the next three lines: a fix for a development-only problem.                                                                                                                                                                                                |
| <pre>const globalForDb = globalThis as unknown as {   pool?: Pool };</pre>                                                                                                        | <code>globalThis</code> is an object that survives when Next.js reloads your files. This line tells TypeScript: "treat it as an object that <i>might</i> have a <code>pool</code> on it". ( <code>as unknown as ...</code> is a TypeScript cast, i.e. "trust me on the type".) |
|                                                                                                                                                                                   |                                                                                                                                                                                                                                                                                |

|                                                                                      |                                                                                                                                                                    |
|--------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>export const pool = globalForDb.pool ?? createPool();</pre>                     | The pool everyone uses. <code>??</code> means "if the left side is empty, use the right side". So: reuse the saved pool if one exists, otherwise create a new one. |
| <pre>if (process.env.NODE_ENV !== "production") {   globalForDb.pool = pool; }</pre> | While developing (not in production), save the pool on <code>globalThis</code> , so after the next reload the line above finds it and reuses it.                   |

## Deep dive: what is a connection pool?

### IN PLAIN ENGLISH

Opening a connection to PostgreSQL is slow: the network handshake plus logging in with a password. And Postgres only accepts about **100** connections at once by default. If every request opened its own connection, the app would be slow, and under load it would run out.

A **pool** is like a taxi rank. It keeps a few connections (taxis) open. Each query takes a free one, uses it, and puts it back. If all are busy, the next query waits a moment in line.

what `pool.query()` does for you

```
pool.query("SELECT ...")
1. borrow a free connection (or open one, up to 10)
2. send the SQL to PostgreSQL
3. wait for the answer
4. give the connection back (closed if unused for ~10 s)
5. hand you the result: { rows: [...] }
```

**Why the `globalThis` trick?** In development, Next.js re-runs a file each time you save it. Without the trick, every save would create a *new* pool while the old ones keep their connections open. After a few dozen saves Postgres says `sorry, too many clients already`. Parking the pool on `globalThis`, which survives reloads, means there is always just one. In production files aren't reloaded, so the trick isn't needed there.

### IF YOU KNOW MONGOOSE / EXPRESS

This file is your `mongoose.connect(process.env.MONGODB_URI)`. Mongoose also keeps a pool behind the scenes; `pg` just makes it visible.

## 12 Write the model: `models/todo.ts`

Create a folder `models` and this file. It's the **only** file that contains SQL. Each function does one job:

| models/todo.ts                                                                                                                                                    |                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Code                                                                                                                                                              | What it means                                                                                       |
| <pre>import { pool } from "@lib/db";</pre>                                                                                                                        | Bring in the shared pool from <code>lib/db.ts</code> . <code>@/</code> means the project root.      |
| <pre>// The "model": everything that talks to the // todos table lives here. // (Like a Mongoose model, but the table's // shape is defined in schema.sql.)</pre> | Comments: this file is the only place that writes SQL. Every other file just calls these functions. |
|                                                                                                                                                                   |                                                                                                     |

|                                                                                                                                                                                  |                                                                                                                                                                                                                                                                                                                                               |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>// Shape of one row in the todos table export type Todo = {   id: number;   text: string;   done: boolean; };</pre>                                                         | <p>A TypeScript <b>type</b> describing one row of the table. It matches <code>schema.sql</code> column by column. Other files import it so TypeScript knows what a todo looks like.</p>                                                                                                                                                       |
| <pre>// SELECT = read rows export async function findAllTodos(): Promise&lt;Todo[]&gt; {</pre>                                                                                   | <p><code>findAllTodos()</code> : returns every todo. <code>async</code> = it takes time (talking to the database). <code>Promise&lt;Todo[]&gt;</code> = "later, you'll get an array of Todos".</p>                                                                                                                                            |
| <pre>  const { rows } = await pool.query&lt;Todo&gt;(     "SELECT * FROM todos ORDER BY id");</pre>                                                                              | <p>Send SQL to Postgres and <code>await</code> (wait for) the answer. <code>&lt;Todo&gt;</code> tells TypeScript what each row looks like. The result object has a <code>rows</code> array; <code>{ rows }</code> pulls it out. <code>ORDER BY id</code> = oldest first.</p>                                                                  |
| <pre>    return rows;   }</pre>                                                                                                                                                  | <p>Give the array back to whoever called this function.</p>                                                                                                                                                                                                                                                                                   |
| <pre>// INSERT = add a row. \$1 is a safe placeholder for the value. export async function createTodo(text: string): Promise&lt;Todo&gt; {</pre>                                 | <p><code>createTodo(text)</code> : adds one row and returns it.</p>                                                                                                                                                                                                                                                                           |
| <pre>  const { rows } = await pool.query&lt;Todo&gt;(     "INSERT INTO todos (text) VALUES (\$1)     RETURNING *",     [text]   );</pre>                                         | <p><code>\$1</code> is a <b>placeholder</b>; the real value comes from the array <code>[text]</code>. Postgres keeps value and SQL separate, so a user typing SQL can't break your query (this is called preventing <b>SQL injection</b>). <code>RETURNING *</code> = send back the row you just made, including its new <code>id</code>.</p> |
| <pre>    return rows[0];   }</pre>                                                                                                                                               | <p>There's exactly one new row, so return the first item.</p>                                                                                                                                                                                                                                                                                 |
| <pre>// UPDATE = change a row. Returns undefined if no row has that id. export async function updateTodoDone(id: number, done: boolean): Promise&lt;Todo   undefined&gt; {</pre> | <p><code>updateTodoDone(id, done)</code> : changes the done flag of one row.</p>                                                                                                                                                                                                                                                              |
| <pre>  const { rows } = await pool.query&lt;Todo&gt;(     "UPDATE todos SET done = \$1 WHERE id = \$2     RETURNING *",     [done, id]   );</pre>                                | <p><code>\$1</code> gets <code>done</code>, <code>\$2</code> gets <code>id</code> (same order as the array). <code>WHERE id = \$2</code> = only touch that one row.</p>                                                                                                                                                                       |
| <pre>    return rows[0];   }</pre>                                                                                                                                               | <p>Return the updated row. If no row had that id, <code>rows</code> is empty, so this returns <code>undefined</code> (the API turns that into a 404).</p>                                                                                                                                                                                     |
| <pre>// DELETE = remove a row export async function deleteTodo(id: number): Promise&lt;void&gt; {</pre>                                                                          | <p><code>deleteTodo(id)</code> : removes one row. <code>Promise&lt;void&gt;</code> = it returns nothing.</p>                                                                                                                                                                                                                                  |
| <pre>  await pool.query("DELETE FROM todos WHERE id = \$1", [id]); }</pre>                                                                                                       | <p>Run the DELETE and wait for it to finish.</p>                                                                                                                                                                                                                                                                                              |

## IF YOU KNOW MONGOOSE / EXPRESS

This is your `Todo` model. Same jobs, different words:

### Mongoose, for comparison only

```
// models/Todo.js (Mongoose)
const todoSchema = new Schema({
  text: { type: String, required: true },
  done: { type: Boolean, default: false },
});
const Todo = model("Todo", todoSchema);

Todo.find().sort({ _id: 1 });
Todo.create({ text });
Todo.findByIdAndUpdate(id, { done }, { new: true });
Todo.findByIdAndDelete(id);
```

### PostgreSQL (this project)

```
-- schema.sql (run once in psql)
CREATE TABLE todos (
  id SERIAL PRIMARY KEY,
  text TEXT NOT NULL,
  done BOOLEAN NOT NULL DEFAULT FALSE
);

SELECT * FROM todos ORDER BY id
INSERT INTO todos (text) VALUES ($1) RETURNING *
UPDATE todos SET done = $1 WHERE id = $2
RETURNING *
DELETE FROM todos WHERE id = $1
```

The big difference: Mongoose's schema lives in your **code**. A Postgres schema lives **inside the database** (you made it in step 5), and the database itself rejects bad data. Our `Todo` type just mirrors it for TypeScript.

## 13 Write the API routes (your controllers)

### IN PLAIN ENGLISH

An **API route** is a URL that returns data instead of a page. In Next.js you create one by making a file called `route.ts`; its **folder path** becomes the URL, and each exported function named after an HTTP method handles that method. No router setup and no `app.listen()`: `npm run dev` already serves it.

Our four endpoints:

| Method + URL               | Send (body)                       | Get back                        | File                        |
|----------------------------|-----------------------------------|---------------------------------|-----------------------------|
| <b>GET</b> /api/todos      | nothing                           | 200 + list of todos             | app/api/todos/route.ts      |
| <b>POST</b> /api/todos     | <code>{"text": "Buy milk"}</code> | 201 + the new todo (or 400)     |                             |
| <b>PATCH</b> /api/todos/7  | <code>{"done": true}</code>       | 200 + updated todo (or 400/404) | app/api/todos/[id]/route.ts |
| <b>DELETE</b> /api/todos/7 | nothing                           | 204, empty                      |                             |

Each handler does three things: **read** the request, **check** it, **call the model** and reply. Create the folders `app/api/todos/` and `app/api/todos/[id]/` (yes, with square brackets in the folder name).

app/api/todos/route.ts

| Code                                                   | What it means                                                                       |
|--------------------------------------------------------|-------------------------------------------------------------------------------------|
| <pre>import { NextResponse } from "next/server";</pre> | <code>NextResponse</code> is how you build the reply: JSON data plus a status code. |



|                                                                                                                                                  |                                                                                                                                                                             |
|--------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>import { createTodo, findAllTodos } from "@/models/todo";</pre>                                                                             | Bring in the two model functions this file needs.                                                                                                                           |
| <pre>// GET /api/todos -&gt; list all todos export async function GET() {</pre>                                                                  | Because this function is <b>named</b> <code>GET</code> , Next.js runs it for <code>GET /api/todos</code> . The URL comes from the folder path <code>app/api/todos/</code> . |
| <pre>  const todos = await findAllTodos();</pre>                                                                                                 | Ask the model for all todos (it runs the SELECT).                                                                                                                           |
| <pre>  return NextResponse.json(todos); }</pre>                                                                                                  | Send them to the browser as JSON. Status is 200 (OK) by default.                                                                                                            |
| <pre>// POST /api/todos body: { text } -&gt; create a todo export async function POST(req: Request) {</pre>                                      | Runs for <code>POST /api/todos</code> . <code>req</code> is the incoming request.                                                                                           |
| <pre>  const { text } = await req.json();</pre>                                                                                                  | Read the JSON the browser sent, e.g. <code>{ "text": "Buy milk" }</code> , and take out <code>text</code> .                                                                 |
| <pre>  if (typeof text !== "string"    !text.trim())   {     return NextResponse.json({ error: "text is required" }, { status: 400 });   }</pre> | <b>Validate</b> before touching the database: if <code>text</code> isn't a string, or is only spaces, reply <b>400 Bad Request</b> and stop.                                |
| <pre>  const todo = await createTodo(text.trim());</pre>                                                                                         | Ask the model to INSERT it (with spaces trimmed off the ends).                                                                                                              |
| <pre>  return NextResponse.json(todo, { status: 201 }); }</pre>                                                                                  | Reply with the new todo and status <b>201 Created</b> .                                                                                                                     |

| app/api/todos/[id]/route.ts                                                                                                               |                                                                                                                                                                                                                                                          |
|-------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Code                                                                                                                                      | What it means                                                                                                                                                                                                                                            |
| <pre>import { NextResponse } from "next/server";</pre>                                                                                    | Same reply helper.                                                                                                                                                                                                                                       |
| <pre>import { deleteTodo, updateTodoDone } from "@/models/todo";</pre>                                                                    | The two model functions this file needs.                                                                                                                                                                                                                 |
| <pre>type Ctx = { params: Promise&lt;{ id: string }&gt; };</pre>                                                                          | Describes the 2nd thing Next.js passes in: <code>params</code> , which holds the <code>[id]</code> part of the URL. For <code>/api/todos/7</code> , <code>id</code> is <code>"7"</code> (always a string). It's a Promise, so you <code>await</code> it. |
| <pre>// PATCH /api/todos/:id body: { done } -&gt; update a todo export async function PATCH(req: Request, { params }: Ctx) {</pre>        | Runs for <code>PATCH /api/todos/7</code> . <code>PATCH</code> = "change part of something".                                                                                                                                                              |
| <pre>  const { id } = await params;</pre>                                                                                                 | Wait for <code>params</code> and take out <code>id</code> .                                                                                                                                                                                              |
| <pre>  const { done } = await req.json();</pre>                                                                                           | Read <code>done</code> from the JSON body, e.g. <code>{ "done": true }</code> .                                                                                                                                                                          |
| <pre>  if (typeof done !== "boolean") {     return NextResponse.json({ error: "done must be true or false" }, { status: 400 });   }</pre> | Validate: must be exactly <code>true</code> or <code>false</code> , otherwise <b>400</b> .                                                                                                                                                               |
| <pre>  const todo = await updateTodoDone(Number(id), done);</pre>                                                                         | Ask the model to UPDATE. <code>Number(id)</code> turns the text <code>"7"</code> into the number <code>7</code> .                                                                                                                                        |
| <pre>  if (!todo) {     return NextResponse.json({ error: "not found" }, { status: 404 });   }</pre>                                      | No row had that id: reply <b>404 Not Found</b> .                                                                                                                                                                                                         |

|                                                                                                                                                          |                                                                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>return NextResponse.json(todo);</code><br><code>}</code>                                                                                           | Success: send back the updated todo.                                                                                                                                                         |
| <code>// DELETE /api/todos/:id -&gt; remove a todo</code><br><code>export async function DELETE(_req: Request, {</code><br><code>params }: Ctx) {</code> | Runs for <code>DELETE /api/todos/7</code> . <code>_req</code> starts with <code>_</code> to show we don't use it (we still need it, because <code>params</code> is always the 2nd argument). |
| <code>const { id } = await params;</code>                                                                                                                | Get the id from the URL.                                                                                                                                                                     |
| <code>await deleteTodo(Number(id));</code>                                                                                                               | Ask the model to DELETE that row.                                                                                                                                                            |
| <code>return new NextResponse(null, { status: 204</code><br><code>});</code><br><code>}</code>                                                           | Reply <b>204 No Content</b> : "done, nothing to send back".                                                                                                                                  |

## Try the API before building the page

With `npm run dev` running, open **`http://localhost:3000/api/todos`** in the browser: you'll see `[]`, the JSON for an empty list. To send other methods, use PowerShell:

PowerShell

```
Invoke-RestMethod -Method Post -Uri http://localhost:3000/api/todos -ContentType "application/json"
-Body '{"text":"Buy milk"}'
Invoke-RestMethod http://localhost:3000/api/todos
Invoke-RestMethod -Method Patch -Uri http://localhost:3000/api/todos/1 -ContentType
"application/json" -Body '{"done":true}'
Invoke-RestMethod -Method Delete -Uri http://localhost:3000/api/todos/1
```

### TIP

Use `Invoke-RestMethod` in PowerShell instead of `curl`: in Windows PowerShell, `curl` is a different command and the JSON quotes get mangled.

## 14 Write the Zustand store: `store/todoStore.ts`

### IN PLAIN ENGLISH

A **store** is a shared whiteboard for the browser side of the app. It holds the todo list and the actions that change it. Any component can read it with `useTodoStore()`, and when the list changes, every component using it redraws.

With one page you *could* keep everything in `useState`. The store pays off when a second component (say, a counter in a header) needs the same list: no passing props down through layers. It also moves the `fetch` calls out of the page, so the page only handles display.

`store/todoStore.ts`

| Code                                                   | What it means                                                                                                                                            |
|--------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>import { create } from "zustand";</code>         | <code>create</code> is Zustand's one main function. It builds a store.                                                                                   |
| <code>import type { Todo } from "@models/todo";</code> | Reuse the <code>Todo</code> type from the model. <code>import type</code> brings in only the type, never the database code, so it's safe in the browser. |

|                                                                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                    |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>// What the store holds: data (todos) + actions (functions that change it) type TodoStore = {   todos: Todo[];   loadTodos: () =&gt; Promise&lt;void&gt;;   addTodo: (text: string) =&gt; Promise&lt;void&gt;;   toggleTodo: (todo: Todo) =&gt; Promise&lt;void&gt;;   removeTodo: (id: number) =&gt; Promise&lt;void&gt;; };</pre> | <p>The TypeScript type of the store: one piece of data ( <code>todos</code> ) and four <b>actions</b> (functions). <code>() =&gt; Promise&lt;void&gt;</code> = "a function that does async work and returns nothing".</p>                          |
| <pre>// create() makes a hook. Any component can call useTodoStore() to read or change it. // set = replace part of the state; get = read the current state.</pre>                                                                                                                                                                       | <p>Comments on the next line.</p>                                                                                                                                                                                                                  |
| <pre>export const useTodoStore = create&lt;TodoStore&gt; ((set, get) =&gt; ({</pre>                                                                                                                                                                                                                                                      | <p>Build the store. You get two helpers: <code>set</code> (change the data, which re-draws any component using it) and <code>get</code> (read the current data). The result, <code>useTodoStore</code>, is a <b>hook</b> that components call.</p> |
| <pre>  todos: [],</pre>                                                                                                                                                                                                                                                                                                                  | <p>Starting data: an empty list.</p>                                                                                                                                                                                                               |
| <pre>  loadTodos: async () =&gt; {     const res = await fetch("/api/todos");     set({ todos: await res.json() });   },</pre>                                                                                                                                                                                                           | <p>Action: GET the list from the API, then <code>set({ todos: ... })</code>. Every component showing <code>todos</code> re-draws automatically.</p>                                                                                                |
| <pre>  addTodo: async (text) =&gt; {</pre>                                                                                                                                                                                                                                                                                               | <p>Action: add a todo.</p>                                                                                                                                                                                                                         |
| <pre>    await fetch("/api/todos", {       method: "POST",       headers: { "Content-Type": "application/json" },       body: JSON.stringify({ text }),     });</pre>                                                                                                                                                                    | <p>Send a <b>POST</b> request. <code>headers</code> says "the body is JSON"; <code>JSON.stringify</code> turns <code>{ text }</code> into a JSON string.</p>                                                                                       |
| <pre>    await get().loadTodos();   },</pre>                                                                                                                                                                                                                                                                                             | <p>Then reload the list from the database, so the screen shows exactly what's saved.</p>                                                                                                                                                           |
| <pre>  toggleTodo: async (todo) =&gt; {</pre>                                                                                                                                                                                                                                                                                            | <p>Action: flip done/not done for one todo.</p>                                                                                                                                                                                                    |
| <pre>    await fetch(`/api/todos/\${todo.id}`, {       method: "PATCH",       headers: { "Content-Type": "application/json" },       body: JSON.stringify({ done: !todo.done }),     });</pre>                                                                                                                                           | <p>Send a <b>PATCH</b> to <code>/api/todos/&lt;id&gt;</code>. The backticks <code>`...\${}`</code> insert the id into the URL. <code>!todo.done</code> = the opposite of its current value.</p>                                                    |
| <pre>    await get().loadTodos();   },</pre>                                                                                                                                                                                                                                                                                             | <p>Reload the list.</p>                                                                                                                                                                                                                            |
| <pre>  removeTodo: async (id) =&gt; {</pre>                                                                                                                                                                                                                                                                                              | <p>Action: delete one todo.</p>                                                                                                                                                                                                                    |
| <pre>    await fetch(`/api/todos/\${id}`, { method: "DELETE" });</pre>                                                                                                                                                                                                                                                                   | <p>Send a <b>DELETE</b> to <code>/api/todos/&lt;id&gt;</code>.</p>                                                                                                                                                                                 |
| <pre>    await get().loadTodos();   }, });</pre>                                                                                                                                                                                                                                                                                         | <p>Reload the list. The <code>});</code> closes the store object, the arrow function and <code>create(</code>.</p>                                                                                                                                 |

## 15 Replace app/layout.tsx

Replace the generated file with this simpler version. (The generated one uses `LayoutProps<"/">`, a newer Next.js helper type; both work.)

| app/layout.tsx                                                                                                 |                                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Code                                                                                                           | What it means                                                                                                                                                    |
| <pre>// layout.tsx wraps every page. It must render &lt;html&gt; and &lt;body&gt;.</pre>                       | Comment: the layout is the frame around every page.                                                                                                              |
| <pre>export const metadata = { title: "To-Do" };</pre>                                                         | Page info for the browser. <code>title</code> is the text on the browser tab.                                                                                    |
| <pre>export default function RootLayout({ children }: { children: React.ReactNode }) {</pre>                   | The layout component. Next.js gives it <code>children</code> (the page being shown). The type says <code>children</code> is anything React can display.          |
| <pre>  return (</pre>                                                                                          | What this component draws.                                                                                                                                       |
| <pre>    &lt;html lang="en"&gt;</pre>                                                                          | The outer HTML tag. Next.js requires the root layout to include <code>&lt;html&gt;</code> and <code>&lt;body&gt;</code> . <code>lang="en"</code> = English page. |
| <pre>      &lt;body style={{ fontFamily: "sans-serif",         maxWidth: 400, margin: "40px auto" }}&gt;</pre> | The page body. The <code>style</code> uses a simple font, max width 400px, and <code>margin: "40px auto"</code> centers it.                                      |
| <pre>        {children}</pre>                                                                                  | The current page ( <code>page.tsx</code> ) is placed here.                                                                                                       |
| <pre>      &lt;/body&gt;     &lt;/html&gt;</pre>                                                               | Closing tags.                                                                                                                                                    |
| <pre>  ); }</pre>                                                                                              | End of what's drawn, end of the function.                                                                                                                        |

## 16 Replace app/page.tsx

The page only **displays** things and **calls store actions**. It never talks to the API or the database directly.

| app/page.tsx                                                                                                                                           |                                                                                                                                                   |
|--------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Code                                                                                                                                                   | What it means                                                                                                                                     |
| <pre>"use client"; // runs in the browser: needed for hooks and click handlers</pre>                                                                   | This component runs in the <b>browser</b> . Needed because it uses hooks and click handlers. (Without it, Next.js runs components on the server.) |
| <pre>import { useEffect, useState } from "react";</pre>                                                                                                | Two built-in React hooks: <code>useState</code> (remember a value) and <code>useEffect</code> (run code after the page shows).                    |
| <pre>import { useTodoStore } from "@/store/todoStore";</pre>                                                                                           | Our Zustand store hook.                                                                                                                           |
| <pre>export default function Home() {</pre>                                                                                                            | The page. Next.js shows it at <code>/</code> because it's <code>app/page.tsx</code> .                                                             |
| <pre>  // Shared state + actions come from the   Zustand store   const { todos, loadTodos, addTodo,   toggleTodo, removeTodo } = useTodoStore();</pre> | Grab the todo list and the four actions from the store. When the store's <code>todos</code> change, this page re-draws.                           |
|                                                                                                                                                        |                                                                                                                                                   |

|                                                                                                                                                                                                                               |                                                                                                                                                                                                                                       |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>// Local state: only this page cares what's typed in the box const [text, setText] = useState("");</pre>                                                                                                                 | <p><code>text</code> = what's typed in the box. Only this page needs it, so plain <code>useState</code> is enough. <b>Rule of thumb:</b> used by one component = <code>useState</code> ; shared data = store.</p>                     |
| <pre>// Load todos from the database once, when the page opens useEffect(() =&gt; {   loadTodos(); }, [loadTodos]);</pre>                                                                                                     | <p>After the page first appears, call <code>loadTodos()</code> . The <code>[loadTodos]</code> list means "run again only if this changes", and it never does, so it runs once.</p>                                                    |
| <pre>async function handleAdd() {</pre>                                                                                                                                                                                       | <p>Runs when you click Add or press Enter.</p>                                                                                                                                                                                        |
| <pre>  if (!text.trim()) return;</pre>                                                                                                                                                                                        | <p>Ignore empty input.</p>                                                                                                                                                                                                            |
| <pre>  await addTodo(text);</pre>                                                                                                                                                                                             | <p>Call the store action (which POSTs to the API and reloads the list).</p>                                                                                                                                                           |
| <pre>  setText(""); }</pre>                                                                                                                                                                                                   | <p>Clear the input box.</p>                                                                                                                                                                                                           |
| <pre>return (</pre>                                                                                                                                                                                                           | <p>Below is JSX: HTML-like code that describes the screen.</p>                                                                                                                                                                        |
| <pre>  &lt;main&gt;     &lt;h1&gt;To-Do&lt;/h1&gt;</pre>                                                                                                                                                                      | <p>A container and a heading.</p>                                                                                                                                                                                                     |
| <pre>    &lt;input       value={text}       onChange={(e) =&gt;         setText(e.target.value)}       onKeyDown={(e) =&gt; e.key === "Enter" &amp;&amp;         handleAdd()}       placeholder="New task..."     /&gt;</pre> | <p>The text box. <code>value={text}</code> shows the state. <code>onChange</code> saves each keystroke into the state. <code>onKeyDown</code> : Enter calls <code>handleAdd()</code> . <code>placeholder</code> = grey hint text.</p> |
| <pre>    &lt;button onClick={handleAdd}&gt;Add&lt;/button&gt;</pre>                                                                                                                                                           | <p>The Add button.</p>                                                                                                                                                                                                                |
| <pre>    &lt;ul&gt;</pre>                                                                                                                                                                                                     | <p>A bullet list.</p>                                                                                                                                                                                                                 |
| <pre>      {todos.map((t) =&gt; (</pre>                                                                                                                                                                                       | <p><code>{ }</code> = "JavaScript here". <code>.map</code> turns each todo into one <code>&lt;li&gt;</code> .</p>                                                                                                                     |
| <pre>        &lt;li key={t.id}&gt;</pre>                                                                                                                                                                                      | <p>One item. <code>key</code> = a unique id so React can track items. Use the database id.</p>                                                                                                                                        |
| <pre>          &lt;input type="checkbox" checked=             {t.done} onChange={() =&gt; toggleTodo(t)} /&gt;</pre>                                                                                                          | <p>Shows <code>done</code> . Clicking calls the store's <code>toggleTodo</code> .</p>                                                                                                                                                 |
| <pre>          &lt;span style={{ textDecoration:             t.done ? "line-through" : "none" }}&gt;{t.text}&lt;/span&gt;</pre>                                                                                               | <p>The task text, crossed out if done.</p>                                                                                                                                                                                            |
| <pre>          &lt;button onClick={() =&gt;             removeTodo(t.id)}&gt;x&lt;/button&gt;</pre>                                                                                                                           | <p>Delete button. <code>() =&gt; ...</code> wraps the call so it runs on click, not immediately.</p>                                                                                                                                  |
| <pre>        &lt;/li&gt;       )}}</pre>                                                                                                                                                                                      | <p>End of one item; end of <code>.map</code> .</p>                                                                                                                                                                                    |
| <pre>    &lt;/ul&gt;   &lt;/main&gt; ); }</pre>                                                                                                                                                                               | <p>Close the list and container; end of the component.</p>                                                                                                                                                                            |

## 17 Run and test

```
PowerShell
```

```
npm run dev
```

Open **http://localhost:3000**, then check:

1. Type a task and press Enter. It appears and the box clears.
2. Tick it. It's crossed out.
3. **Refresh the page.** It's still there, because it came back from PostgreSQL.
4. Click **x**. It's gone.
5. Look in the database directly: `psql -U postgres -h localhost -d todo_app -c "SELECT * FROM todos;"`

# Part 5: Express + Mongoose vs Next.js + PostgreSQL

Same ideas, different places.

| Job                     | Express + Mongoose                                         | This project                                                                             |
|-------------------------|------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Start the server        | <code>app.listen(3000)</code> in <code>server.js</code>    | <code>npm run dev</code> (Next.js is the server)                                         |
| Connect to the database | <code>mongoose.connect(uri)</code>                         | <code>new Pool({ connectionString })</code> in <code>lib/db.ts</code>                    |
| Define the data shape   | <code>new Schema({...})</code> in code                     | <code>CREATE TABLE</code> in <code>schema.sql</code> , run once + <code>Todo</code> type |
| Model / queries         | <code>Todo.find()</code> , <code>Todo.create()</code> ...  | Functions in <code>models/todo.ts</code> running SQL                                     |
| Routes                  | <code>router.get("/todos", ctrl.list)</code>               | Folder <code>app/api/todos/</code> + exported <code>GET</code>                           |
| Controllers             | <code>(req, res) =&gt; res.json(...)</code>                | <code>GET(req)</code> returning <code>NextResponse.json(...)</code>                      |
| URL parameter           | <code>req.params.id</code>                                 | <code>[id]</code> folder + <code>await params</code>                                     |
| Request body            | <code>req.body</code> (needs <code>express.json()</code> ) | <code>await req.json()</code>                                                            |
| IDs                     | <code>_id : ObjectId string</code>                         | <code>id : 1, 2, 3...</code> (SERIAL)                                                    |
| Shell                   | <code>mongosh</code>                                       | <code>psql</code>                                                                        |
| GUI                     | MongoDB Compass                                            | pgAdmin 4                                                                                |
| Frontend                | separate React app + CORS setup                            | same project, same address; no CORS                                                      |

## Express + Mongoose, for comparison only

```
// Express: routes + controller
router.post("/todos", async (req, res) => {
  const { text } = req.body;
  if (!text?.trim())
    return res.status(400).json({ error: "text is required" });
  const todo = await Todo.create({ text });
  res.status(201).json(todo);
});
```

## this project (shortened)

```
// Next.js: app/api/todos/route.ts
export async function POST(req: Request) {
  const { text } = await req.json();
  if (typeof text !== "string" || !text.trim())
    return NextResponse.json({ error: "text is required" }, { status: 400 });
  const todo = await createTodo(text.trim());
  return NextResponse.json(todo, { status: 201 });
}
```

## IN PLAIN ENGLISH

**Do I need Express?** No. Next.js route handlers already do Express's job (routing, reading requests, sending JSON), and they run on Node.js. You'd add Express only if you wanted a separate backend server.

**Want Mongoose-style models with Postgres?** That's what an **ORM** is. **Prisma** and **Drizzle** are the popular ones: you describe tables in code and write `db.todo.findMany()` instead of SQL. Learn raw SQL first (this project); ORMs make more sense once you know what they generate.

## Part 6: Memorize and practice

### The journey of one click

What happens when you type "Buy milk" and press Enter:

1. `page.tsx`: `onKeyDown` sees Enter and calls `handleAdd()`.
2. `handleAdd()` calls the store action `addTodo("Buy milk")`.
3. `todoStore.ts`: `fetch("/api/todos", { method: "POST", body: '{"text":"Buy milk"}' })`.
4. The request crosses from browser to server. `Next.js` finds `app/api/todos/route.ts` and runs `POST`.
5. `POST` reads the JSON, checks `text`, calls `createTodo("Buy milk")`.
6. `models/todo.ts`: `pool.query("INSERT ... VALUES ($1) RETURNING *", ["Buy milk"])`.
7. The pool lends a connection; PostgreSQL saves the row and returns `{id: 1, text: "Buy milk", done: false}`.
8. `POST` replies **201** with that JSON.
9. Back in the store: `loadTodos()` does a GET and calls `set({ todos })`.
10. React sees the store changed and redraws the list. "Buy milk" appears.

### One table to rule them all

| Action | Page                   | Store action            | HTTP                               | Route function | Model function              | SQL    |
|--------|------------------------|-------------------------|------------------------------------|----------------|-----------------------------|--------|
| Read   | <code>useEffect</code> | <code>loadTodos</code>  | GET <code>/api/todos</code>        | GET            | <code>findAllTodos</code>   | SELECT |
| Create | <code>handleAdd</code> | <code>addTodo</code>    | POST <code>/api/todos</code>       | POST           | <code>createTodo</code>     | INSERT |
| Update | <code>checkbox</code>  | <code>toggleTodo</code> | PATCH <code>/api/todos/:id</code>  | PATCH          | <code>updateTodoDone</code> | UPDATE |
| Delete | <code>x button</code>  | <code>removeTodo</code> | DELETE <code>/api/todos/:id</code> | DELETE         | <code>deleteTodo</code>     | DELETE |

### Rules to remember

- `app/` folder path = URL. `page.tsx` = page. `route.ts` = API. `[id]` = changing part.
- Exported function name in `route.ts` = HTTP method.
- `"use client"` = runs in the browser. Needed for hooks and clicks. Database code must **never** be in a client file.
- Only the model writes SQL. Routes check input and call the model. The page only calls the store.
- SQL values always go in `$1`, `$2` placeholders, never glued into the string.
- Secrets go in `.env.local`, read as `process.env.NAME`, on the server only. Restart after changing.
- One component needs it: `useState`. Many components share it: the store.
- One pool for the whole app; `pool.query()` borrows and returns connections for you.

### Practice ladder (do them in order)

1. **Rebuild from memory.** Delete `lib`, `models`, `store`, `app/api` and `page.tsx`, then rebuild using only the table above.
2. **Count.** Show "2 of 5 done" above the list. Compute it from `todos` in the page (no new state).



3. **Filter (Zustand).** Add `filter: "all" | "active" | "done"` and a `setFilter` action to the store; add three buttons. Browser-only, no API change.
4. **Created date (SQL).** In `psql`: `ALTER TABLE todos ADD COLUMN created_at TIMESTAMP NOT NULL DEFAULT now();` then add it to the `Todo` type and show it.
5. **Edit text (full stack).** Model `updateTodoText`, accept `text` in `PATCH`, a store action, and an Edit button.
6. **Clear completed.** `DELETE FROM todos WHERE done = true`: a model function, a `DELETE /api/todos` handler, a store action and a button.
7. **Errors and loading.** Add `loading` and `error` to the store; check `res.ok` after each fetch; show "Loading..." and error messages.
8. **Optimistic update.** In `toggleTodo`, flip it in the store *first*, then call the API; undo if it fails.
9. **Try an ORM.** Rebuild `models/todo.ts` with Prisma or Drizzle and compare.

## Troubleshooting

| You see                                                                                         | Fix                                                                                                                                                    |
|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>psql</code> is not recognized                                                             | Add <code>C:\Program Files\PostgreSQL\17\bin</code> to PATH (step 3) and open a <b>new</b> terminal.                                                   |
| password authentication failed for user "postgres"                                              | The password in <code>.env.local</code> (or what you typed) is wrong. Restart <code>npm run dev</code> after fixing.                                   |
| <code>ECONNREFUSED 127.0.0.1:5432</code>                                                        | PostgreSQL isn't running. Run <code>sc.exe query postgresql-x64-17</code> ; start it in the Services app.                                              |
| database "todo_app" does not exist                                                              | Do step 4.                                                                                                                                             |
| relation "todos" does not exist                                                                 | Do step 5, and make sure you used <code>-d todo_app</code> .                                                                                           |
| sorry, too many clients already                                                                 | Something creates many pools. Make sure only <code>lib/db.ts</code> calls <code>new Pool</code> .                                                      |
| Cannot find module '@/...'                                                                      | Check the file path and the <code>paths</code> entry in <code>tsconfig.json</code> .                                                                   |
| An error saying <code>useState</code> / <code>useEffect</code> only works in a Client Component | Add <code>"use client";</code> as the first line of the page.                                                                                          |
| Page loads but the list stays empty                                                             | Open <code>http://localhost:3000/api/todos</code> . If it shows an error, read the <code>npm run dev</code> terminal; the real error is printed there. |

Generated from the project's real source files by `guide/build-guide.mjs`. The `guide/` folder is not part of the app.